Informe técnico · Configuración del ambiente de desarrollo (Módulo 1)

Juan Manuel Torres Erazo · Especialización en Inteligencia Artificial
Universidad Autónoma de Occidente · Computación en la nube · 9 de noviembre de 2026

1. Propósito y contexto

La práctica se montó en mi portátil personal con Windows: dos máquinas virtuales creadas desde un archivo de texto (Vagrantfile), un servidor que guarda la base SITECALI-Res y un cliente que la consulta por una red privada. Luego empaqueté el servidor como box de Vagrant, la publiqué en un repositorio en la nube y le cargué la base con una regla automática sobre qué puede salir. Así se cumple lo que pide la actividad (Vagrant y VirtualBox, una box publicada en la nube y ejercicios Linux), sobre los datos reales del proyecto.

SITECALI-Res integra más de mil variables sobre las 15.658 manzanas de Cali, y parte de sus fuentes es sensible: registros de homicidios, lesionados y emergencias con lugar y fecha, y datos educativos por estudiante. Publicados como puntos, expondrían a personas concretas, varias de ellas víctimas. Dado lo anterior, el proyecto necesita un lugar aislado donde procesar la base completa y una regla de divulgación aplicada por el sistema y verificable por un tercero.

Leído desde la definición del NIST (SP 800-145), el ambiente reproduce a escala local la infraestructura como servicio: el hipervisor entrega cómputo, memoria, disco y red virtualizados, y sobre ellos se decide el sistema operativo y lo demás. El servidor opera como una nube privada de un solo usuario, con autoservicio (las máquinas se crean con un comando), recursos asignados y acceso por red. Lo que sale por la compuerta es lo que después irá a Azure. Hay dos cambios frente a la clase: CentOS 7 perdió soporte el 30 de junio de 2024 y se usó Ubuntu 24.04; Vagrant Cloud dejó de aceptar boxes el 1 de octubre de 2026 y la box se publicó en GitHub.

2. Arquitectura del ambiente

El ambiente tiene dos capas de aislamiento: la máquina virtual frente a Windows y, dentro del servidor, la zona interna frente a la pública. VirtualBox 7.2.22 es el hipervisor de tipo 2. Vagrant 2.4.9 crea las máquinas desde el Vagrantfile versionado en el repositorio: infraestructura como código. El servidor usa Ubuntu 24.04.3 (box bento 202510.26.0, versión fijada) con 4 GB y 4 vCPU,

PostgreSQL 16.15, PostGIS 3.4.2, GDAL 3.8.4 y Python 3.12 con geopandas y pyarrow. El cliente tiene 1 GB y solo el cliente psql.

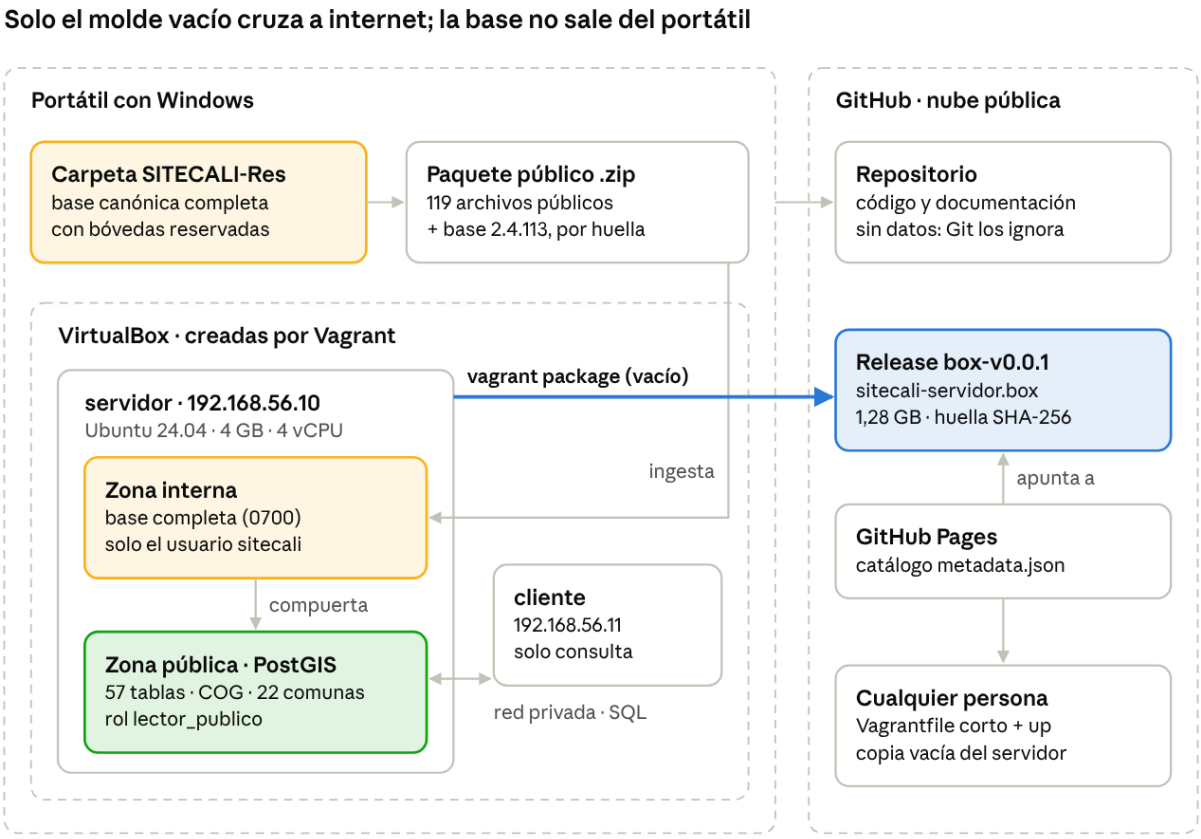


Figura 1 · arquitectura del ambiente y lo que llega a GitHub

Cada máquina tiene una interfaz NAT para salir a internet, idéntica en ambas (10.0.2.15) porque VirtualBox aísla una NAT por máquina, y una red privada solo-anfitrión (servidor 192.168.56.10, cliente .11) por donde se comunican. La guía usa los nombres servidorUbuntu y clienteUbuntu y el rango 192.168.100.x; aquí las máquinas se nombran por la función que cumplen y no por su sistema operativo, que ya cambió una vez de CentOS a Ubuntu, y usan el rango 192.168.56.x, el que VirtualBox asigna por defecto a las redes solo-anfitrión, de modo que el mismo Vagrantfile funciona también en macOS y Linux sin editar el archivo networks.conf. PostGIS se reenvía solo a 127.0.0.1:15432 del propio portátil. La frontera entre zonas la imponen los permisos del sistema de archivos: la zona interna (0700) solo la lee el usuario de servicio sitecali, sin acceso interactivo, y la pública (0755) se expone en el esquema publico de PostgreSQL a un único rol de red, lector_publico, de solo lectura y con clave generada dentro de la máquina.

El servidor ve el repositorio por un directorio sincronizado: Vagrant monta una carpeta de Windows dentro de la máquina con las carpetas compartidas de VirtualBox (tipo vboxsf), que dependen de las adiciones de invitado ya incluidas en la box de bento, y por eso no se usó el plugin vagrant-vbguest. Se desactivó el montaje por defecto en /vagrant y se montó la raíz del repositorio en /sitecali-repo en solo lectura, de modo que la máquina ejecuta el código versionado pero no puede alterarlo. La prueba lo muestra en ambos sentidos: un archivo escrito en Windows se leyó al instante desde el servidor, y el intento de escribir desde el servidor recibió «Read-only file system» (captura 42). La base nunca se abre desde allí: la ingesta la copia al disco de la máquina, porque SQLite, y con él GeoPackage, es lento sobre esa carpeta y depende de bloqueos de archivo que ella no garantiza.

3. Proceso de instalación y publicación de la box

El proceso quedó documentado en una bitácora de 69 registros y 74 capturas que aquí se citan por número. La preparación instaló Git 2.56, VirtualBox 7.2.22 y Vagrant 2.4.9, la primera versión que reconoce VirtualBox 7.2 (capturas 00–09). Como la carpeta de usuario tiene un espacio en el nombre, se definió VAGRANT_HOME=C:\vagrant.d y el proyecto se ubicó en C:\SITECALI. El servidor arrancó con 4 GB mediante una variable de entorno (captura 10).

La réplica de la clase 1 empezó con vagrant up, que descargó la box base, creó las dos máquinas con sus adaptadores y las aprovisionó en unos tres minutos (capturas 11–14). En cada una se revisó el sistema con hostnamectl, se instalaron net-tools y vim con apt y se vieron las interfaces con ifconfig. El ping del cliente al servidor devolvió 4 de 4 paquetes, a 0,75 ms y con ttl=64, es decir, sin enrutadores de por medio (capturas 15–20).

La réplica de la clase 2 partió de una restricción: una box copia el disco entero, por lo que antes de empaquetar se borró la clave del lector y se comprobó que las carpetas de datos y de credenciales estaban vacías (captura 21). vagrant package produjo una box de 1,28 GB, por debajo del límite de 2 GiB de GitHub, que se registró con vagrant box add (capturas 22–23) y se publicó como versión box-v0.0.1 del repositorio (capturas 25–28). Su catálogo metadata.json, con su huella SHA-256, cumple el papel de la página de Vagrant Cloud. La huella que calculó GitHub coincidió con la de Get-FileHash (captura 24). Desde otra carpeta, un Vagrantfile de cuatro líneas descargó la box, verificó su huella y levantó una máquina con PostgreSQL activo y sin datos ni claves (capturas 29–29k). Al final,

vagrant provision regeneró la clave del lector e instaló pyarrow, porque el GDAL de Ubuntu no lee Parquet (captura 29g).

Git se usó desde Windows y no desde el servidor, como propone la guía, para que las credenciales de GitHub nunca entren a una máquina que se empaqueta entera como box. El servidor, que ya trae Git 2.43, descarga el repositorio público con git clone sin credenciales (captura 43).

Problema	Causa	Solución
Vagrant no reconoció el catálogo (29a–29c)	Exige application/json; GitHub raw da text/plain	Catálogo en GitHub Pages
vagrant ssh pedía contraseña (29e, 29f, 29i)	El SSH de Windows descarta llaves con permisos abiertos	icacls, y /reset para borrarla
Clave del lector rechazada (34a, 34b)	El portapapeles traía 42 caracteres, no 32	Clave en una variable

4. Funcionamiento: ingesta, compuerta y ejercicios Linux

La ingesta cargó la base en 2 minutos y 40 segundos (capturas 30–32), publicó 57 tablas en PostGIS y no dejó salir nada interno: la auditoría no tuvo hallazgos y pasaron las 15 pruebas de la compuerta (captura 33). Usa dos fuentes verificadas por huella: la base 2.4.113 congelada y el paquete público de la base canónica, la nueva arquitectura de SITECALI-Res en GeoParquet y GeoTIFF. Ese paquete lo arma en Windows un script que copia solo los 119 archivos marcados como públicos y bloquea por ruta las bóvedas reservadas.

Lo público atraviesa tres puertas:

1. La primera es la herramienta de la base (sitecali.py marco --publico), que entrega solo indicadores públicos.
2. La segunda es una lista de permitidos: de la canónica sale cada archivo solo si su huella coincide con el manifiesto, y cada columna solo si el catálogo la marca como pública; salieron 108 archivos, se retuvieron 11 y se quitaron 15 columnas (capturas 32a y 32b).
3. La tercera transforma: los 7.578 homicidios de 2018 a 2024 salen como 968 celdas de una rejilla adaptativa, con el 99,6 % de los eventos representado y ninguna celda con menos de cinco casos ($k \geq 5$). Una auditoría que no reutiliza ese código revisó 40 tablas y 795 columnas

contra 14 prefijos internos. Las pruebas intentaron pedir los puntos de homicidio, colar un archivo reservado, alterar la herramienta y bajar k a 2; las 15 fueron rechazadas.

Desde el cliente, con el rol de solo lectura, la consulta devolvió 15.658 manzanas, 13.999 con población, y 2.216.858 habitantes en 2018 y 2.274.837 en 2024 (captura 34). GDAL convirtió los rásteres de edificación a GeoTIFF optimizado para la nube (COG) y recortó el de 2023 en las 22 comunas (captura 35).

Los ejercicios básicos de la guía de Linux (pwd y cd, mkdir -p, rmdir, pushd y popd, touch, cp, mv, cat, more y rm) se corrieron en el servidor con un script que imprime cada comando antes de ejecutarlo y al final borra lo creado (capturas 44a y 44b). A la pregunta de la guía: more muestra el texto página por página, casi solo hacia adelante, y se cierra al llegar al final; less permite avanzar y retroceder, buscar con / y abrir archivos grandes sin leerlos completos, y no se cierra hasta presionar q. Sobre esa base, los ejercicios aplicados usaron el servidor real:

Tema (captura)	Comandos	Resultado
Permisos (36)	ls -la, id, sudo -u	vagrant recibe «Permission denied» en la zona interna
Disco y memoria (37a)	df, free, du, unzip -l	Zona interna (772 MB) y zona pública (775 MB)
Huellas y texto (37b, 38)	sha256sum, grep, tail	La misma huella que en Windows; rechazos en la bitácora
Servicios y red (39)	systemctl, ps, ip, ss	PostgreSQL activo; puertos 22 y 5432
SIG (40a, 40b)	ogrinfo, ogr2ogr	Comunas de EPSG:9377 a GeoJSON geográfico

5. Resultados, aprendizajes y lo que sigue

Queda un ambiente reproducible desde un archivo de texto, una box que cualquiera puede descargar y verificar, y un nodo que procesa la base completa y deja salir solo lo público, todo en el repositorio juanmatorreserazo/sitecali-res-cloud. Entiendo que una imagen reutilizable se convierte en un vehículo de fuga, así que empaquetar antes de cargar datos y sin claves es una decisión de seguridad y responsabilidad.

Una compuerta que confía en una clasificación hereda sus errores, y por eso necesita auditoría independiente, pruebas que intenten romperla y la regla de cerrarse ante lo que no sabe clasificar: cuando la canónica sumó durante la práctica un catálogo que la ingesta aún no lee, nueve archivos

quedaron retenidos en vez de publicarse sin verificar. Y buena parte del trabajo real está en condiciones que los tutoriales no muestran, que se resolvieron entendiendo su causa.

Un aprendizaje clave fue que montar una nube, aunque quepa en un portátil, es decidir dónde viven los datos de una ciudad, quién puede verlos y bajo qué reglas salen. En una base que guarda la huella de homicidios, víctimas y estudiantes, esa decisión recae sobre personas concretas. Por eso la práctica no se dio por buena solo porque las máquinas encendieran, sino porque la frontera entre lo interno y lo público resistiera pruebas diseñadas para romperla. Al final, la infraestructura debe responder a una pregunta: ¿cómo hacer legible la desigualdad de Cali, manzana por manzana, sin exponer a quienes la viven?

Referencias

GDAL. *COG – Cloud Optimized GeoTIFF generator*. gdal.org

GitHub. *About releases*. GitHub Docs. docs.github.com

HashiCorp. *Box repository*. Vagrant Documentation. developer.hashicorp.com

HashiCorp. *HCP Vagrant deprecation: important dates and migration guidance*. hashicorp.com

HashiCorp. *Synced folders: basic usage*. Vagrant Documentation. developer.hashicorp.com

Lautzenhiser, M. (2024). *CentOS Linux has reached its End of Life (EOL)*. Red Hat. redhat.com

Mell, P. y Grance, T. (2011). *The NIST Definition of Cloud Computing* (NIST SP 800-145). National Institute of Standards and Technology. doi.org/10.6028/NIST.SP.800-145